

VIETNAM NATIONAL UNIVERSITY – HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



WEB APPLICATION DEVELOPMENT

Final Project Report

Topic: Hotel Management System

By Group 36 – Member List

1	Nguyễn Phước Đước	ITCSIU23005	Team Leader
2	Lê Trọng Vinh	ITCSIU23045	Team Member

Instructor: Assoc. Prof. N.T.Nghia

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
LIST OF FIGURES.....	3
ABSTRACT.....	4
CHAPTER 1: INTRODUCTION.....	5
1.1 System Overview.....	5
1.1.1. Project goals.....	5
1.1.2. Techniques and tools used:.....	5
1.1.3. Scope and limitation.....	6
1.2 Report Structure:.....	6
CHAPTER 2: TASK TIMELINE & DIVISION.....	7
2.1 Project Timeline and Task Allocation Table.....	7
2.2 Contribution breakdown.....	10
CHAPTER 3: METHODOLOGY.....	11
3.1 Database design.....	11
3.1.1. ER Diagram.....	11
3.1.2. Database schema diagram.....	13
3.2 Application/Website Structure.....	14
3.2.1 Project Architecture.....	14
3.2.2 Class Structure.....	14
3.2.3 Database Connection Implementation.....	14
3.2.4 Graphical User Interface (GUI) Design.....	15
CHAPTER 4: IMPLEMENTATION & DISCUSSION.....	16
4.1. Implementation Process.....	16
4.1.1. Database Setup.....	16
4.1.2. Backend Development.....	16
4.1.3. Frontend Development.....	16
4.1.4. Connecting Frontend and Backend.....	16
4.1.5. Testing and Fixing Issues.....	16
4.1.6. Final Adjustments and Documentation.....	16
4.2. Results & Analysis.....	17
CHAPTER 5: CONCLUSION & RECOMMENDATION.....	19
5.1 Summary of Key Findings.....	19
5.2 Reflection on Objectives.....	19
5.3 Implications.....	19
5.4 Future Work.....	19
REFERENCES.....	21

LIST OF FIGURES

Table 2.1 Project Timeline and Task Allocation.....	7
Table 2.2 Member Contribution Breakdown.....	10
Figure 3.1 Entity Relationship Diagram.....	11
Figure 3.2 Database Schema Diagram.....	13

ABSTRACT

Hotels often have difficulty manual management such as customer information and dispersed rooms, monitoring inaccurate room status, complex payment process and lack of a centralized database system. These problems reduce operational efficiency and negatively affect customer experience. The Hotel Management System project is implemented with the goal of building a relational database to support the hotel to operate effectively and scientifically. The system focuses on core functions including monitoring room status, booking management, customer information storage and staff, as well as payment processing on a unified foundation. Through the application of important principles in database management such as standardization, reference integrity and indexing techniques, the data guarantee project is accurate, quickly and reliable. The expected result is a hotel management system capable of improving operational efficiency, minimizing errors in information processing and supporting managers' decision-making process.

CHAPTER 1: INTRODUCTION

1.1 System Overview

In today's industry world, the hotels have to manage an enormous volume of customers everyday, resulting in longer time of checking-in and low-quality services. Some hotels started to engage in a digital approach to address the problems. Traditional processes such as: manual or semi-manual processes are still widely used by majority hotels which can lead to delay, low-accuracy records. These issues negatively affect the guests' experience and the hotel operations which emphasize the need of a more reliable and effective system. The Hotel management system is designed to automatically assist customers and employees in managing and analyzing the database of the Hotel system. The system provides a platform where staff can observe the room status, booking management, and customer information. Furthermore, the clients are able to book faster and easier.

1.1.1. Project goals

The objective of the Hotel management system is to provide excellent services for customers and employees by enhancing the management operations. This system concentrates on optimizing the operations, which plays an integral part of improving the customer's experience.

The aim of this project is to replace the traditional management by the new system which gives a better efficiency and saves time, reducing the mistakes, errors and also capable of handling hotels from a distance.

1.1.2. Techniques and tools used:

The development of the Hotel Management System utilizes a combination of tools and technologies across different layers:

- Database Layer: MySQL.
- Interface Layer: HTML, CSS^[6].
- Backend Processing Layer: Java Spring Boot^[2].
- Development and Source Control: Visual Studio Code, Git, and GitHub.

Rationale for Technology Selection

MySQL was selected as the main database because it is stable, industry-standard, and works perfectly with Java-based enterprise applications. **Spring Boot** was chosen as the core framework because it eliminates external web server administration through its embedded Apache Tomcat container. This allows the master application to run as an independent, standalone execution context initialized directly via

`HotelManagementApplication.main()`, dramatically lowering environment friction and setup complexity.

For the frontend, we used HTML and CSS to build a clean and easy-to-use interface so users can navigate the system without difficulty. Visual Studio Code was used as our main IDE because it's lightweight, user-friendly, and efficient, and Git/GitHub supported our version control and teamwork throughout the development process.

1.1.3. Scope and limitation

The Hotel Management System provides a unified platform, where customers can search for empty rooms, make reservations, use the accompanying services and send questions directly to the hotel. After the booking is created, the staff will manage the detailed booking, store customer records, process payment and ensure service quality. Hotel management can monitor all activities, customer data and services, as well as evaluate operational efficiency. This process ensures the support system and interact with customers and internal management effectively and integrated.

However, the system still has some limitations. First, it requires the access of the internet for operation, and the system can be overloaded during the peak time. In addition, Staff training is essential to ensure the performance and proper usage of the system features.

1.2 Report Structure:

This report is divided into five chapters.

- Chapter 1: Introduces the system, project goals, tools, scope, and a brief outline of the report.
- Chapter 2: Presents the project timeline and explains how tasks were divided among team members.

- Chapter 3: Describes the methods used, including database design, data insertion, queries, and (if applicable) the application structure.
- Chapter 4: Discusses the implementation process, results, and challenges faced during the project.
- Chapter 5: Concludes the report and suggests possible future improvements.
- References: Introducing all documents and resources used during the development and documentation process.

CHAPTER 2: TASK TIMELINE & DIVISION

2.1 Project Timeline and Task Allocation Table

STAGE	WEEK	ACTIVITIES	MEMBER	DEADLINE
Planning and Early Analysis	1	Select the topic, outline the basic direction of project construction and development, and the roles of project members	All Members	3/5
		Research references, and technical documents for selecting the best technique and methodology for project management	Vinh	
		Unify the technique and methodology used in the project and research technical documents to use those technologies	All members	
	2	Create a development	Vinh	3/5

		timeline for the entire project		
		Draw simple ER diagrams to visualize ideas about the database of the project	Được	
		Sketch the relational table	Được	
Conceptual Design	3	Determine the most prevalent use cases and actors in the system	Được	3/5
		Specify the sorts of information that must be saved in the database	Được	
		Prepare the project proposal	Vinh	
		Review and retrospective	All members	3/5
Realization of Design	4 - 12	Database creation and configuration	Được	12/5
		Create database tables, their relationships, restrictions, and class diagram	Được	12/5
		Implement the queries required for each functionality	Được	12/5
		Design and develop UI/UX and interactive	Vinh	8/5

		interface for the website of the project		
		Link the MySQL database to the website	All Members	17/5
		Midterm report, Review and retrospective	All members	17/5
Technical Debt and Final Testing	13 - 14	Refactoring and code review	Được	31/5
		Detection and debugging of bugs	Được	
		Design and execute test cases	Được	
		Fix and change the code of the project until it is stable	Trọng Vinh	
		Final report and design slides and presentation script	Trọng Vinh	
		Review and retrospective	All Members	
Presentation	15	Presentation Project	All Members	3/6

Table 2.1 Project Timeline and Task Allocation.

2.2 Contribution breakdown

Contributor	Contribution Heading	Contribution Rate
Lê Trọng Vinh ITCSIU23045	Frontend UI layout creation (HTML/CSS), Client and Admin view router routing architecture (ViewController), Frontend-Backend endpoint testing, and Technical documentation.	50%
Nguyễn Phước Được ITCSIU23005	Core Backend Architecture design, Standalone Spring Boot Environment configuration, Spring Data JPA layer implementation, Database Schemas, and REST API/DTO endpoints deployment.	50%

Table 2.2 Member Contribution Breakdown.

CHAPTER 3: METHODOLOGY

3.1 Database design

3.1.1. ER Diagram

The Entity–Relationship Diagram (ERD) illustrated in Figure 3.1. represents a Hotel Management System that manages hotels, employees, rooms, guests, reservations, and payments. It aims to ensure data consistency and support efficient hotel operations such as booking, check-in/check-out, and billing. The design follows standard database normalization and ER modeling principles^[5].

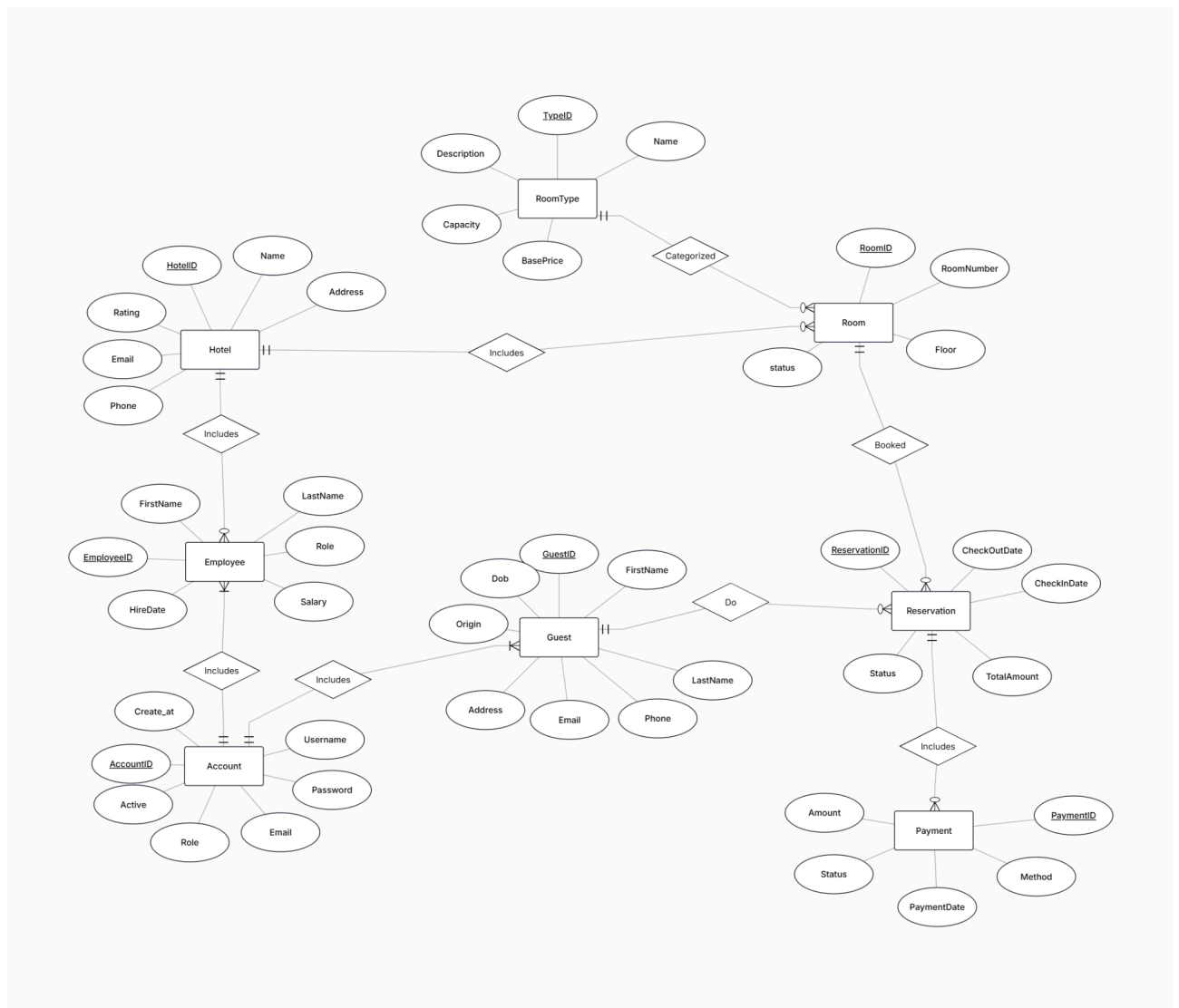


Figure 3.1 Entity Relationship Diagram.

Main Entities and Attributes

- Hotel: HotelID, Name, Rating, Address, Phone, Email – represents each hotel branch.
- Employee: id, dob, idNumber, hotel, account, FirstName, LastName, Phone, HireDate, Salary, Position – staff details.
- Account: id, Username, password, Role, Email, active, createdAt – login and authorization data.
- Room: id, roomType, hotel, RoomNumber, Floor, Status, Note – represents hotel rooms.
- RoomType: id, Name, Description, Capacity, basePrice – defines room categories.
- Guest: id, FirstName, email, dob, idNumber, account, LastName, Address, Origin, Phone – stores guest information.
- Reservation: id, CheckIn, CheckOut, totalAmount, guest, hotel, inChargeEmployee, Status – booking records.
- Payment: id, Amount, Method, reservation, status, paymentDate – payment transactions.

Key Relationships

- One Hotel includes many Employees and Rooms.
- Each Employee and Guest is associated with one Account.
- Each Room is categorized to one RoomType.
- A Guest can make multiple Reservations, and each reservation associated with one Payment.
- the ReservationRoom associative entity, It resolves the many-to-many relationship between reservations and rooms and contains attributes: id, reservation, room, and price.

Strengths and Improvements:

The design is modular, normalized, and supports role-based access. It covers essential hotel operations comprehensively. Future enhancements may include Services entities for extended functionality.

3.1.2. Database schema diagram

The database schema illustrated in Figure 3.2. represents the logical implementation of the Hotel Management System. It defines the structure of the database through tables, attributes, primary keys, and foreign keys that establish relationships among entities such as hotels, employees, rooms, guests, reservations, and payments. Each table specifies appropriate data types (e.g., varchar, integer, datetime) to ensure data integrity and accuracy. The schema also includes associative tables like ReservationRoom to handle many-to-many relationships between reservations and rooms. Overall, this schema translates the conceptual ERD into a relational model that supports efficient data management and enforcement of referential integrity within the hotel system.

The Hotel Management System Schema shows the relationships between entities to each other. The Hotel Management System database is designed to manage hotel operations, guest bookings, and payments. The Hotel entity includes name, address, email, phone, and star rating, connecting with rooms and employees. Rooms are possessed by a specific hotel and categorized under a Roomtype. In addition, Guests can make reservations for rooms. Reservation has one or many payments. Guests and Employees have their own Accounts for authentication and role management.

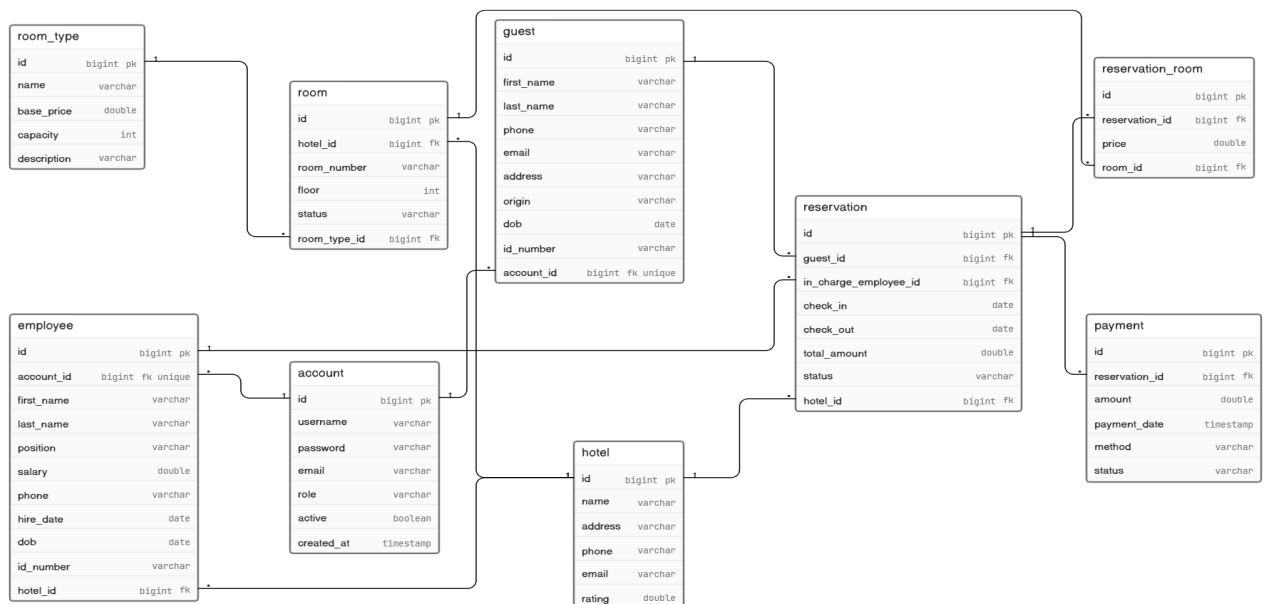


Figure 3.2 Database Schema Diagram.

3.2 Application/Website Structure

3.2.1 Project Architecture

The system implements a decoupled corporate architecture utilizing **Spring Boot** as a hybrid backend application engine. It separates responsibilities into two major paradigms:

- **REST Data Layer:** Handled via `@RestController` configurations that ingest and emit clean JSON data streams across HTTP channels for asynchronous state mutations.
- **View Router Layer:** Governed by a centralized `@Controller` routing interface (`ViewController`) that maps logical web request keys straight to frontend static assets (`index`, `client`, `hotels`, `reservation`, `authentication`, `guest`).

3.2.2 Class Structure

- To isolate core database persistence layers from network communications, the system enforces a strict **Data Transfer Object (DTO)** architectural design pattern. Core tables are never exposed directly to HTTP consumers. Instead, objects like `RoomDTO`, `EmployeeDTO`, and `ReservationDTO` aggregate multiple complex internal relationship joins—flattening related foreign schemas (e.g., nesting `List<ReservationRoomDTO>` directly inside a parent `ReservationDTO`) into a streamlined payload that significantly optimizes network transit and prevents circular serialization references.

3.2.3 Database Connection Implementation

The persistence infrastructure relies on a direct **Spring Data JPA / Hibernate** bridge communicating with a target MySQL database instance. Database connection criteria—including JDBC driver routing, connection pool configurations, and security credentials—are declaratively injected through the application's runtime environment properties (`application.properties`). Hibernate dynamically manages the lifecycle of connection states and executes mapping mutations without needing external middleware execution platforms.

3.2.4 Graphical User Interface (GUI) Design

The frontend is built with HTML and CSS^[5], focusing on simplicity and clear navigation. Each page contains forms, buttons, and tables depending on the required functionality. Spring Boot^[2] served as static web assets or routed through the view controller configuration.

CHAPTER 4: IMPLEMENTATION & DISCUSSION

4.1. Implementation Process

The project was implemented step by step, starting with the database and ending with the full system integration.

4.1.1. Database Setup

The MySQL^[2] database was created using MySQL Workbench^[1]. Tables were built based on the ERD, and the connection to Spring Boot^[2] was configured through the application.properties file.

4.1.2. Backend Development

The backend was implemented in Spring Boot^[1] using controllers, services, repositories, entities, and DTOs. Each feature was built by creating the necessary endpoints and connecting them to the database.

4.1.3. Frontend Development

The frontend pages were created using HTML and CSS. These pages included forms, tables, and simple layouts that matched the system's functions served as static web assets or routed through the view controller configuration .

4.1.4. Connecting Frontend and Backend

The frontend forms were linked to backend APIs, allowing data to be sent and received. This made the pages fully functional, such as viewing records or adding new entries.

4.1.5. Testing and Fixing Issues

After connecting everything, the team tested each part of the system. Errors such as wrong mappings, missing fields, or connection issues were fixed during this stage.

4.1.6. Final Adjustments and Documentation

Once the system was working smoothly, final adjustments were made and the documentation was prepared.

4.2. Results & Analysis

After completing the implementation, the system was tested to confirm that all main features worked as expected. The connection between the frontend, backend, and database are functioned smoothly, and data could be inserted, updated, retrieved, and displayed without errors.

During testing, the team verified several key points:

- Controllers correctly processed requests and returned the right responses.
- Services applied the expected logic.
- Repositories interacted with the database without issues.
- The frontend forms successfully communicated with the backend APIs.
- The overall workflow—from entering data to displaying results—worked as intended.

Overall, the final system met the project's initial objectives. The structure was stable, the data flow was correct, and the main functions of the application were delivered as planned.

Challenges & Solutions

During development, several challenges came up, especially while connecting the backend, frontend, and database. These issues were identified and resolved one by one.

Challenge 1: Team Coordination and Leadership Issues

During the early stages of the project, the absence of consistent leadership led to a lack of clear vision and project management. This resulted in miscommunication among team members, unclear task distribution, and delays in aligning the database design with backend implementation.

Solution:

To address this issue, the team reorganized responsibilities and clarified development objectives. The database structure and entity classes were carefully reviewed and synchronized with the backend logic. After correcting mismatched fields and updating the database connection settings, the backend was successfully integrated with MySQL [3], allowing the project to proceed smoothly and efficiently.

Challenge 2: Backend–Database Integration Issues

During the initial boots of the standalone application using `SpringApplication.run()`, runtime errors occurred due to structural differences between the primitive database column attributes in MySQL and the advanced Java data types defined within the entity layers (e.g., handling Java `LocalDate` for employee birthdays and timestamps for payments).

Solution:

The database structure and entity classes were reviewed and corrected. After fixing mismatched fields and updating the connection settings, the backend connected to MySQL^[3] without issues.

Challenge 3: Frontend–Backend Communication

When connecting the HTML/CSS pages with Spring Boot^[2] APIs, the JSON responses became nested due to entity relationships, creating circular references the frontend couldn't process.

Solution:

DTOs were introduced, controller endpoints were adjusted, and unnecessary mappings were removed. After updating the frontend forms to match the new API structure, data flow worked correctly.

CHAPTER 5: CONCLUSION & RECOMMENDATION

5.1 Summary of Key Findings

The Hotel Data Management System transitioned the hotel's operations from a manual, legacy system to a unified, high-performance digital platform.

The key findings indicate the achievements:

1. Efficiency: Reduced the average reservation time, boosting front-desk productivity.
2. Accuracy: ensured reliability for room inventory and financial reporting.
3. Real-Time Business Intelligence: Management can make timely, data-driven decisions on pricing and staffing based on the data given by the system.

5.2 Reflection on Objectives

A better efficiency and saves time system: Achieved. Core task processing times were minimized.

- Reducing the mistakes, errors: Achieved. The system acts as a single, reliable source of truth.
- Remote Management: Achieved. Since the system is deployed on a web-based, managers can monitor data from anywhere with an internet connection.
- Guest pages: completed but missing feature. Since the system already has the page for guests to create reservations but it's still a static page.

5.3 Implications

The hotel management system emphasizes the value of data centralization within the hospitality sector. It demonstrates how modern technology can create a powerful, cost-effective, and flexible solution to traditional management systems.

5.4 Future Work

Guest pages Feature Implementation: Due to lack of development time, guest features will be implemented in the near future.

Cloud Optimization: Changing to a database service to reduce maintenance overhead and improve auto-scaling capabilities.

AI pricing: Implement an AI to analyze data and forecast demand, enabling the system to automatically adjust room price.

REFERENCES

- [1] VMware, “Spring Boot Reference Documentation.” Available:
<https://spring.io/projects/spring-boot>
- [2] Oracle Corporation, “MySQL Documentation.” Available: <https://dev.mysql.com/doc/>
- [3] Oracle Corporation, “Java Persistence API Documentation.” Available:
<https://jakarta.ee/specifications/persistence/>
- [4] R. Elmasri and S. B. Navathe, “Fundamentals of Database Systems,” Pearson Education.
- [5] Mozilla Developer Network, “HTML & CSS Documentation.” Available:
<https://developer.mozilla.org/>